

HINHJ: Hierarchical Attention-Based Heterogeneous Graph Neural Network for DNS Hijacking Detection

Haoran Jiao¹, Cong Dong¹, Chenglong Li¹, Jiahai Yang¹, *Senior Member, IEEE*, Leyao Nie, Yuling Liu², Jing Wang, Changzhi Zhao³, and Xia Yin⁴

Abstract—The Domain Name System (DNS) is a critical internet infrastructure that translates human-readable domain names into machine-routable IP addresses. However, DNS is inherently vulnerable to manipulation, with hijacking attacks growing in both frequency and sophistication. Existing detection methods primarily rely on traffic analysis at specific network points. However, they suffer from limited coverage and low accuracy in complex environments, such as when CDN is employed. While recent approaches employ graph-based techniques, they still suffer from detection inaccuracy issues due to their failure to account for the complex interdependencies among multiple types of nodes. To address these limitations, we propose a novel heterogeneous graph-based detection framework. Based on the collected DNS records from distributed scanners, our method extracts activity and security features and constructs a heterogeneous graph to capture resolution patterns and cross-entity relationships. We further design a time-decay graph neural network TNHAN that enhances traditional Heterogeneous Graph Attention Networks (HAN) by dynamically weighting recent records. This network improves adaptability to legitimate DNS changes. For evaluation, we conduct experiments on real-world resolvers and domain datasets. Experiment results demonstrate the effectiveness of our method. Our method can achieve an F1-score of 0.96, outperforming the best baseline by 0.057 on average, and up to 0.113 under low label proportion. Moreover, we conduct several case studies on detected incidents, including cases related to geopolitical conflicts, censorship-related hijacking, and manipulation by malicious resolvers. These cases demonstrate the method's effectiveness in identifying diverse hijacking behaviors in practice.

Index Terms—DNS hijacking, graph neural network, heterogeneous graph, hierarchical attention mechanism.

I. INTRODUCTION

THE Domain Name System (DNS) serves as the cornerstone of the entire internet, functioning as the critical bridge between human-readable domains and machine-routable IP addresses. However, DNS inherently possesses numerous security vulnerabilities that render it susceptible to manipulation and tampering [1]. DNS hijacking has grown in both frequency and complexity in recent years. It now includes not only malicious activities but also state-level censorship and commercial exploitation. This form of DNS compromise not only leads to data breaches but also poses significant risks to the stability of critical infrastructure systems. Hence, it is significant to develop effective detection mechanisms specifically targeting DNS hijacking.

Notably, the DNS hijacking studied in this paper refers to the destruction of DNS integrity. DNS integrity is defined as users consistently receiving unaltered responses from authoritative servers [1]. It can be violated through two attack vectors: (1) Institutional manipulation via query interception or response forgery by network operators [2], [3], [4]; (2) Technical exploits like cache poisoning by malicious actors [5], [6], [7]. Recursive resolvers are particularly vulnerable to these integrity violations due to their intermediary role in DNS resolution. As the critical link between end users and authoritative servers, they face threats from both institutional interference and technical exploits.

Detecting DNS hijacking attacks presents a complex challenge due to multiple inherent difficulties. A primary obstacle lies in DNS's extensive attack surface. Its distributed architecture and complex recursive query process involve extensive network communication, creating multiple potential points for hijacking. Another critical difficulty stems from the continuous emergence of vulnerabilities. At the DNS protocol level, persistent cache poisoning vulnerabilities allow malicious records to overwrite legitimate cached entries [6], [7], [8]. Simultaneously, system-level vulnerabilities enable attackers to directly compromise authoritative servers or modify records at domain registrar systems [9].

To address these challenges, current research has proposed various detection approaches. The predominant methodology

Received 4 June 2025; revised 8 December 2025; accepted 11 March 2026. Date of publication 19 March 2026; date of current version 27 March 2026. This work was supported by Zhongguancun Laboratory. The associate editor coordinating the review of this article and approving it for publication was Prof. Libing Wu. (*Corresponding authors: Cong Dong; Chenglong Li.*)

Haoran Jiao, Cong Dong, and Leyao Nie are with Zhongguancun Laboratory, Beijing 100194, China (e-mail: jiaohr@mail.zgclab.edu.cn; dongcong@mail.zgclab.edu.cn; niely@mail.zgclab.edu.cn).

Chenglong Li and Jiahai Yang are with the Institute for Network Sciences and Cyberspace, Tsinghua University, Beijing 100084, China, and also with Zhongguancun Laboratory, Beijing 100194, China (e-mail: lichenglong@tsinghua.edu.cn; yang@cernet.edu.cn).

Yuling Liu and Changzhi Zhao are with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100085, China, and also with the School of Cyber Security, University of Chinese Academy of Sciences, Beijing 101408, China (e-mail: liuyuling@iie.ac.cn; zhaochangzhi@iie.ac.cn).

Jing Wang is with the College of Artificial Intelligence, Beijing Normal University, Beijing 100875, China (e-mail: wangjing@bnu.edu.cn).

Xia Yin is with the Department of Computer Science and Technology, Tsinghua University, Beijing 100084, China, and also with Zhongguancun Laboratory, Beijing 100194, China (e-mail: yxia@tsinghua.edu.cn).

Digital Object Identifier 10.1109/TIFS.2026.3675876

involves deploying traffic capture devices at specific network locations during the resolution process and applying detection methods on the captured traffic. However, these methods exhibit several critical limitations. First, these methods demonstrate limited coverage against attacks targeting different network locations. By relying exclusively on traffic analysis from specific monitoring points, they fail to detect hijacking activities occurring in non-cooperative domains, particularly those downstream relative to the observation points [10], [11], [12], [13], [14]. Second, accuracy issues arise as they rely on single-point features such as NS/AS changes. Consequently, they are only effective for domains with simple deployment structures and fail to identify DNS hijacking in complex scenarios like CDN environments accurately [15], [16], [17]. To enhance accuracy, recent studies employ graph-based methods [18]. However, they only focus on simple node connections rather than detecting abnormal interaction patterns between resolvers, IPs, and domains. This limitation also results in missed detections for sophisticated hijacking attacks.

In this paper, we propose a novel heterogeneous graph-based approach for DNS hijacking detection. Targeting specific public resolvers and domain lists, our method employs geographically distributed scanners to probe and collect DNS records. Based on these records, the approach generates two distinct feature categories, activity features and security features. To comprehensively capture cross-entity relationships in DNS resolution, we construct a heterogeneous graph that visually represents both the resolution process and the complex associative structures of resolution records. Based on this graph, we design a novel graph neural network architecture to model the complex neighbor relationships accurately. Building upon conventional heterogeneous graph neural networks, our model introduces a time-decay neighbor selection mechanism TNHAN that enhances traditional Heterogeneous Graph Attention Networks (HAN). This mechanism dynamically adjusts the weights of resolution records from different dates. In particular, it assigns higher weights to more recent records, thus adapting to scenarios involving legitimate modifications of DNS records.

To validate the effectiveness of the proposed method, we constructed a representative list of resolvers and critical domains. We evaluated our approach using DNS records collected from these targets and conducted comparative experiments with state-of-the-art methods in this field. Experimental results demonstrate that our method achieves superior accuracy, attaining an F1-score of 0.96. Furthermore, based on the detection results, we performed a manual in-depth analysis of several hijacking incidents, which yielded case studies characterizing typical DNS hijacking behaviors.

The main contributions of this paper are as follows:

- This paper proposes a heterogeneous graph framework that models domain-resolver-answer_ip correlations through four predefined relationship types and multi-dimensional node features. This approach dynamically captures evolving DNS resolution patterns, enabling systematic analysis of hijacking behaviors.
- This paper presents a novel Heterogeneous Graph Dual-Layer Attention Neural Network (TNHAN) with a time-decay neighbor selection mechanism for DNS hijacking detection. The proposed network architecture incorporates a time-decay neighbor selection mechanism that enhances conventional HAN, resulting in significant improvements in detection performance.
- We deploy our method in a real-world network environment and successfully obtain multiple cases of different types of DNS hijacking events, with purposes including censorship factors, political factors, and malicious hijacking.

The remainder of this paper is structured as follows. Section II summarizes the preliminary and previous works related to DNS hijacking. Section III elaborates on the details of our proposed HINHJ method. Section IV describes the experiment settings. Section V shows the detailed analysis results of experiment and provides several hijacking case studies in reality. Section VI provides the conclusion.

II. RELATED WORK

As a core internet infrastructure, the domain name system faces security threats from both malicious attacks and manipulation. In 2024, a large-scale DNS hijacking in China exploited router vulnerabilities to redirect millions of users to fake CDN nodes. This incident disrupts services like Alibaba Cloud and Tencent Cloud [19]. State-level interventions routinely replace authentic IP addresses with government-approved servers [14]. For example, Iran's DNS tampering for political censorship. Commercial operators also prefer to hijack user queries to their own DNS services (e.g., 8.8.8.8) [20]. For example, Chinese companies redirect 30% of Google DNS traffic to local servers for cost reduction or ad injection [14]. These practices compromise data security and destabilize critical internet infrastructure.

To mitigate the aforementioned challenges, recent studies on DNS hijacking detection proposed several approaches. They can be categorized into three: response record based methods, traditional machine learning based methods, and graph learning methods.

1) Response Record Based Methods:

DNS hijacking can be detected based on analyzing response inconsistencies to specialized queries. Randall et al. [10] detect router-level interception by sending diagnostic requests to public DNS services, whereas Luo et al. [11] identify hijacked resolvers through subdomain probes monitored at authoritative servers. While these methods require minimal resources, their effectiveness depends on target configurations and cannot detect hijacking at public resolver level. Through experiments with controlled domains, Liu et al. [14] reveal widespread DNS interception by comparing expected and actual query resolution paths. Akiwate et al. [12] combine internet scans with DNS/certificate logs to detect historical anomalies in domain-IP-certificate patterns. Their shortage remains in that manual verification limits scalability. Nosyk et al. [13] passively detect middlebox injections using

a) Stability vs. Volatility					b) Consistency vs. Isolation					
Date	Normal Resolver		Hijacked Resolver		Domain	Normal Resolver1		Normal Resolver2		Hijacked Resolver
2024-12-01	IP1	AS1	IP1	AS1	domain1	IP1	AS1	IP1	AS1	IPx ASx
2024-12-08	IP2	AS1	IP1	AS1	domain2	IP2	AS2	IP2	AS2	IPy ASy
2024-12-15	IP2	AS1	IPx	ASx						
2024-12-22	IP1	AS1	IP1	AS1						

c) Security vs. Risk		
Domain	Normal Resolver	Hijacked Resolver
domain	High reputation IP	Low reputation IP

Fig. 1. Core concepts. Differences between normal and hijacking records can be categorized into three aspects: stability vs volatility, consistency vs isolation, and security vs risk.

root server response patterns, but miss DNS cache poisoning attacks.

2) Traditional Machine Learning Based Methods:

Machine learning approaches for DNS hijacking detection typically combine statistical features with algorithmic analysis. Houser et al. [15] utilize DNSDB data to identify server response anomalies (e.g., IP/NS record changes) through SVM and RF models trained on daily passive DNS features. While effective in detecting new NS/AS ownership patterns, this method shows elevated false positives due to overlooking resolver-domain-IP correlations. Jin et al. [17] develop a multi-dimensional detection framework incorporating protocol features, temporal features, and geographic features to distinguish legitimate DNS updates from malicious alterations. Fejrskov et al. [16] use traffic data to extract multiple types of resolver features, including resolver prefixes, number of associated records, etc., and use machine learning methods to implement DNS hijacking detection. Their method assumes that if a resolver is well-known, it is benign. However, well-known resolvers can also cause DNS hijacking.

3) Graph Learning Methods:

In recent years, graph learning based methods have emerged in multiple anomaly monitoring domains. Its effectiveness in capturing sharp relationships among different nodes for detection makes it a focus in complex detection problems. In this paper, we also follow this routine to design our method. In previous studies related to DNS hijacking, graph-based detection methods address this problem through relational analysis of interconnected network entities. Jia et al. [18] demonstrate this approach by constructing resolver-web server interaction graphs and using Louvain community detection and node2vec [21] embeddings to identify malicious resolvers. However, this method relies only on a bipartite graph between the resolver and the web server. As a result, the relationships it captures are not comprehensive.

III. METHODOLOGY

In this section, we introduce our proposed method for DNS hijacking detection. We first introduce the core concepts that motivated us to design the approach. Then we briefly introduce

the overall architecture of the HINHJ. After that, we elaborate on each process of HINHJ in detail.

A. Core Concepts

In this section, we examine several representative DNS hijacking examples and abstract three core concepts that guide our detection design. As shown in Figure 1, DNS hijacking exhibits abnormal connection patterns among domains, resolvers, and records. These patterns can be directly shown through changes in node relationships on graphs. Specifically, differences between normal and hijacking records can be categorized into three aspects: stability, consistency, and security.

Stability vs volatility. Benign DNS resolution demonstrates structural stability, consistently resolving to static record IP addresses or CDN nodes. In contrast, DNS hijacking induces abrupt topological shifts through recursive resolver cache poisoning. This triggers sudden resolution pattern mutations. As a result, these behavioral contrasts manifest in graph structures as spatial stability versus spatial volatility.

Consistency vs isolation. Benign resolvers demonstrate global consistency. While load balancing and geo-routing strategies [22], [23] may create variations, resolution patterns remain coherent within geographic or network boundaries. In contrast, hijacked resolvers exhibit behavioral isolation when persistently manipulating specific domains for purposes like advertisement injection and censorship. Leveraging graph analysis can capture the distinction that benign resolution patterns form cohesive subgraphs over time, while hijacked resolution patterns appear as topological outliers beyond dominant communities.

Security vs risk. Benign DNS resolutions exhibit high-confidence security attributes, commonly appearing in threat intelligence whitelists. Conversely, hijacking resolutions show strong associations with blocklisted entities and high-risk domains. Leveraging graph analysis can capture the distinction through node attributes that hijacking nodes exhibit low-credibility features and connect to high-risk domains.

B. Overview

As shown in Figure 2, HINHJ comprises four processes: records collection, graph construction, feature design and TNHAN-based detection. HINHJ first collects response messages from recursive resolvers through active probing. Then

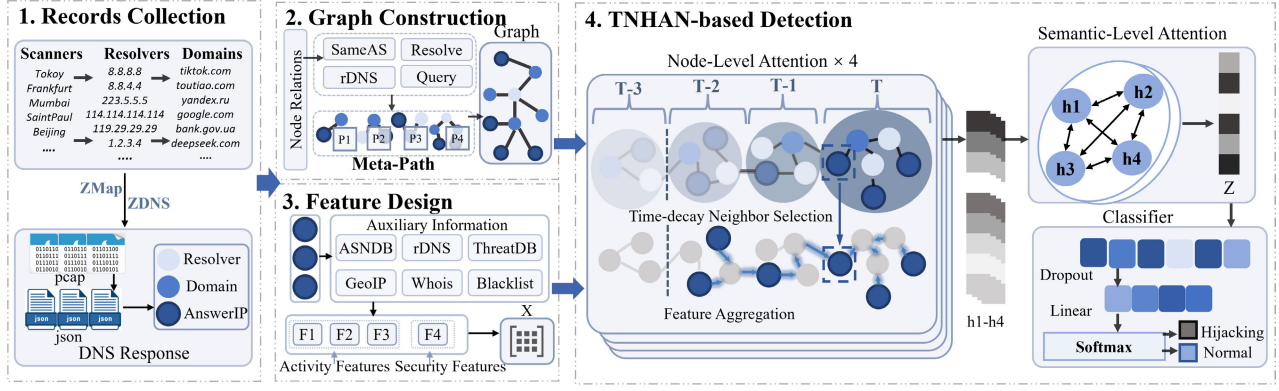


Fig. 2. The overall architecture of HINHJ.

HINHJ constructs a heterogeneous graph and corresponding meta-path set by combining various auxiliary information. The graph and path represent the association relationships among resolvers, domains, and answer_ips (the IP address returned as the DNS answer) in DNS scenarios. Subsequently, HINHJ produces behavior patterns of each entity as features of nodes. Finally, to effectively identify abnormal patterns from heterogeneous graph with properties of meta-path set and node features, the TNHAN model is employed for DNS hijacking detection. The following sections elaborate details of each component.

C. Records Collection

The first process of HINHJ is collecting DNS records from resolvers. In general, collecting these records can be divided into active probing and passive monitoring. Both of these methods can achieve the collection of resolver, domain and records. However, considering the flexibility and the specificity of monitoring targets, we prefer to choose the active probing. Specifically, we maintain two specific lists for monitoring, a critical domain list and a public resolver list. In probing, we initially request the domains on the list to each public resolver and parse corresponding response to collect records. This procedure is repeated periodically to observe the change of each record.

Notably, when PDNS sources from strategic locations are available, the collection process can also be conducted by passive monitoring. These strategic locations include the ingress or egress points of resolvers and critical routing nodes that cover the target resolver's network interactions. Such positions ensure that passive monitoring can obtain rich redundant records, including interested targets. Therefore, HINHJ can also leverage such monitored PDNS data in its subsequent processing.

D. Graph Construction

The DNS resolution process involves data flows across multiple entities. To model the relationships among these entities, we prefer to use a graph structure for representation. Since the types of entities in the resolution process are different, it causes information loss when using a homogeneous graph. This is due to ignoring the relationship among different types

of entities. Hence, we employ the Heterogeneous Information Network (HIN) [24] to capture more semantics and support more comprehensive representations.

As shown in Fig. 2, an HIN is built consisting of resolvers, domains, answer_ips (the IP address returned as the DNS answer). Formally, the heterogeneous graph is defined as $G = \langle V, E \rangle$, with $V = \{V_d, V_r, V_a\}$ denoting domains, resolvers, and answer_ips respectively, and $E = \{E_1, E_2, E_3, E_4\}$ represents four types of relationships. The details of these relations are as follows, and Table I lists their adjacency matrices.

- Resolver *queries* to Domain. We use matrix Q to denote that resolver queries to domain.
- Domain *is resolved to* answer_ip. We use matrix R to denote that domain is resolved to answer_ip.
- *rDNS (reverse DNS record)* of answer_ip contains domain. We use matrix F to denote that the reverse DNS record for answer_ip i contains domain j.
- Resolver *in the same AS of* another resolver. We use Matrix A to denote that two resolvers belong to the same AS.

ASN information and rDNS information of resolvers and records are acquired through open source [25] and host command queries. Among these relations, Q, R, and F represent directional interactions. We model them as directed edges. Specifically, DNS queries originate from resolvers and domains, while resolution mappings and reverse DNS relationships also have clear source and target semantics. Modeling them as directed edges preserves the directionality of information flow in the DNS resolution process. In contrast, A captures the symmetric AS-level co-location between resolvers, so we model it as an undirected relation.

We formulate DNS hijacking detection as an answer_ip node classification task. On the built graph, we define several metapaths to focus on crucial relationships [26]. We specifically consider metapaths where both endpoints are IP nodes, yielding four fundamental metapath types $P = \{P1, P2, P3, P4\}$ as detailed in Table II, where a , d , and r represent the answer_ip, domain, and resolver, respectively.

These meta-paths are designed to capture different semantic relationships between answer_ips. These include co-resolving to the same domain, sharing reverse DNS entries, and being returned by the same or AS-affiliated resolvers. Each meta-

TABLE I
ADJACENCY MATRIX

Matrix	Element	Description
Q	$q_{i,j}$	If domain j is queried by resolver i , then $q_{i,j} = 1$. Otherwise, $q_{i,j} = 0$.
R	$r_{i,j}$	If domain j is resolved to answer_ip i , then $r_{i,j} = 1$. Otherwise, $r_{i,j} = 0$.
F	$f_{i,j}$	If the reverse DNS record for answer_ip i contains domain j , then $f_{i,j} = 1$. Otherwise, $f_{i,j} = 0$.
A	$a_{i,j}$	If resolver i and resolver j belong to the same AS, then $a_{i,j} = 1$. Otherwise, $a_{i,j} = 0$.

TABLE II
META-PATH DEFINITIONS

PID	Meta-Path	Commuting Matrix M	Description
1	$a \xrightarrow{R} d \xrightarrow{R^\top} a$	$RR^\top$	answer_ips resolve to the same domain
2	$a \xrightarrow{F} d \xrightarrow{F^\top} a$	$FF^\top$	answer_ips' reverse DNS records contain the same domain
3	$a \xrightarrow{R} d \xrightarrow{Q} r \xrightarrow{Q^\top} d \xrightarrow{R^\top} a$	$RQQ^\top R^\top$	answer_ips co-occur in records of the same resolver
4	$a \xrightarrow{R} d \xrightarrow{Q} r \xrightarrow{A} r \xrightarrow{Q^\top} d \xrightarrow{R^\top} a$	$RQAQ^\top R^\top$	answer_ips co-occur in records from different resolvers within the same AS

TABLE III
FEATURE DESCRIPTION

Category	Subtype	Feature	Description
Activity Features	Query Frequency	dynamic_activity_index	Dynamic activity index with time decay, higher values indicate recent activity concentration
		longest_active_streak	Maximum consecutive active days reflecting behavioral persistence
	Request Diversity	std_degree_nd	Standard deviation of daily associated domains
		domain_count	Unique domain count per query round
		domain_count_ndays	Total unique domains in the time window
		domain_diversity_entropy	Domain diversity entropy via TLD distribution
	Network Topology	resolver_count	Unique resolver count within two hops
		is_cdn	Whether the answer_ip's AS belongs to CDN providers
Security Features	/	ip_resolver_as_sim	AS similarity between answer_ip and resolvers
		ip_resolver_geo_sim	Geographic similarity between answer_ip and resolvers
		blocklist	Historical block status of associated answer_ip
		sensitive_ratio	Ratio of gambling/pornography domains
		gov_ratio	Ratio of government-affiliated domains
		ttl_long	TTL exceedance flag

path represents a type of semantic connection, and HINHJ applies semantic-level attention to learn the relative importance of each meta-path.

Each meta-path is formally encoded as a commuting matrix. We use these matrices to build a subgraph, connecting answer_ips that meet the meta-path condition. In the node-level attention stage, the model learns to aggregate features across these subgraphs.

The selection of meta-paths is guided by domain knowledge of DNS hijacking. The selected meta-paths reflect different dimensions of relationships between answer_ips, including domain-level association (co-resolving to the same domain), network-level similarity (sharing reverse DNS entries), and resolver-level behavior (being returned by the same or AS-affiliated resolvers). These patterns correspond to key stages in the DNS resolution process and contribute discriminative information for distinguishing between benign and hijacked answer_ips.

E. Node Feature Design

In the heterogeneous graph, each answer_ip node can be correlated with abundant information. As an enhancement, we further produce node features. As shown in Table III, these features are categorized into activity features and security features.

1) *Activity Features*: Activity features are designed to capture the dynamic behavior of answer_ips within a time

window. This type of feature reflects their stable and change pattern over a period. These features include three subtypes: query frequency, request diversity, and network topology.

a) *Query frequency features*: *Dynamic Activity Index (DAI)* is calculated as follows,

$$DAI = \sum_{t=1}^T w(t) \cdot \mathbb{I}(\text{answer_ip active at day } t),$$

$$w(t) = e^{-\lambda(T-t)} \quad (1)$$

where λ is the attenuation coefficient (we set $\lambda = 0.7$), and T is the total number of days in the time window. A higher DAI value indicates recent concentrated activity, whereas a lower value indicates earlier or more evenly distributed activity. *longest_active_streak* represents the answer_ip's maximum consecutive active days within the time window. Higher value indicates sustained activity or persistent DNS hijacking, while lower values suggest potential cache pollution or configuration anomalies. *std_degree_nd* measures daily fluctuations in domain associations, where abnormal volatility may signal malicious hijacking.

b) *Request diversity features*: *Domain_count* and *domain_count_ndays* represent the number of associated domains in a single query round and across time windows respectively. Significant daily variations in these metrics may suggest potential hijacking activity. *resolver_count* quantifies the unique resolvers within two hops of the answer_ip,

while the Domain Diversity Entropy (DDE) measures domain diversity through the following equation,

$$DDE = - \sum_{k=1}^K p_k \log_2 p_k, \quad p_k = \frac{n_k}{N} \quad (2)$$

where K represents the total common TLD types (e.g., .com, .net, .gov), n_k denotes the request count for TLD_k and N is the total request volume.

c) *Network topology features*: The binary feature *is_cdn* identifies whether the answer_ip's autonomous system (AS) belongs to CDN providers, as attackers typically are less likely to point the hijacking to CDN. *ip_resolver_as_sim* checks whether the answer_ip and its resolver (within two hops) share the same AS. A higher value suggests the resolver is resolving domains to its own AS, potentially indicating hijacking. Similarly, *ip_resolver_geo_sim* detects answer_ip-resolver country matches.

2) *Security Features*: Security features are used to describe the security risks of answer_ip itself. The *blocklist* feature identifies whether the answer_ip appears in historical malicious lists, though this feature alone cannot definitively assess risk due to potential answer_ip dynamism. The *sensitive_ratio* feature measures the proportion of sensitive domains associated with the answer_ip, where a higher ratio indicates greater security risk due to potential malicious activity. Similarly, *gov_ratio* quantifies the fraction of government-related domains linked to the answer_ip. Additionally, *ttl_long* identifies DNS records with TTL values exceeding a predefined threshold, as DNS hijacking attacks frequently employ abnormally high TTL values to prolong their effectiveness.

Finally, we construct a feature matrix x . This matrix is used with the heterogeneous graph as input to the later TNHAN model.

F. Attention Network Classifier

To effectively classify answer_ip nodes in heterogeneous graphs, we propose TNHAN, a dual-layer attention neural network based on heterogeneous graphs. It enables HINHJ to not only learn from node features in time series but also to dynamically weigh the importance of different neighborhood structures and semantic meta-paths. This enhances the model to discern critical relational patterns indicative of hijacking, rather than treating all connections as equally significant.

For specific, TNHAN takes the heterogeneous graphs $G = (V, E)$, meta-path set $P = \{p_1, p_2, p_3, p_4\}$ and feature matrix x as input. Briefly, TNHAN includes node-level attention, semantic-level attention, and classification. We elaborate each layer in detail.

1) *Node-Level Attention*: HINHJ constructs a subgraph $\mathcal{G}_k$ for each meta-path p_k , and then calculates the attention weight of answer_ip node i to neighbor node j under meta-path p_k .

$$\alpha_{ij} = \frac{\exp(\text{ReLU}(\mathbf{a}_k^\top [W_k x_i \| W_k x_j]))}{\sum_{j' \in \mathcal{N}_k(i)} \exp(\text{ReLU}(\mathbf{a}_k^\top [W_k x_i \| W_k x_{j'}]))} \quad (3)$$

This is a node-level attention neural network that calculates the importance between neighbor nodes i and j in the subgraph.

$W_k \in \mathbb{R}^{d \times d_h}$ is the projection matrix of the meta-path p_k , and $\mathbf{a}_k \in \mathbb{R}^{2d_h}$ is the attention parameter vector of the meta-path p_k .

It is worth noting that when searching for the neighbors of node i , we do not directly select in the heterogeneous graph, but adopt a time-decayed neighbor selection strategy. By prioritizing more recent interactions and changes in neighborhood patterns, HINHJ is more sensitive to the abrupt behavioral shifts that often characterize the onset of hijacking events, an aspect that static graph analyses might overlook.

$$\mathcal{N}_k^{(t)}(i) = \mathcal{N}_k(i) \cup \left\{ j \in \mathcal{V}^{(t)} \setminus \mathcal{N}_k(i) \mid \sum_{\tau=t-\Delta}^{t-1} \mathbb{I}(j \in \mathcal{N}_k^{(\tau)}(i)) \cdot \lambda^{t-\tau} \geq \theta \right\} \quad (4)$$

$\mathcal{N}_k(i)$ denote the neighbor set of node i in the daily sub-graph $\mathcal{G}_k$, where $\mathcal{V}$ represents the complete node set of $\mathcal{G}_k$. The time window Δ filters recent active neighbors, and the threshold θ selects the most active neighbors. The decay factor $\lambda \in [0, 1)$ controls the influence of past interactions, where a smaller λ assigns lower weights to older neighbor occurrences. This exponential decay ensures that more recent interactions contribute more to the final neighbor selection, effectively capturing temporal locality in graph dynamics. The time-aware neighbor set $\mathcal{N}_k^{(t)}(i)$ captures node interactions within Δ .

After that, the neighbor features are aggregated through Path-Specific Aggregation:

$$h_{i,k} = \sigma \left(\sum_{j \in \mathcal{N}_k^{(t)}(i)} \alpha_{ij}^k W_k x_j \right) + W_{\text{res}} x_i \quad (5)$$

In 5, $W_{\text{res}} \in \mathbb{R}^{d \times d_h}$, σ is the ReLU activation function. This step uses node-level attention calculation to identify which neighbor answer_ip under each meta-path is more important to the target node, and finally generates the node embedding matrix H_k under the meta-path p_k .

2) *Semantic-Level Attention*: After aggregating information within each meta-path, it remains to combine the resulting embeddings to form a unified node representation. Since different meta-paths capture different semantics, their contributions to the final classification may vary. Semantic-level attention is used to adaptively weigh the importance of each meta-path based on its relevance to the task. Following node-level attention, HINHJ produces node embeddings H_k for each meta-path p_k . The model then dynamically integrates these semantic representations. First, our method learns the global context vector and its importance weight for each meta-path:

$$c_k = \text{MEAN}(H_k) \in \mathbb{R}^{d_h} \quad (6)$$

$$\gamma_k = \mathbf{v}^\top \tanh(U c_k + b) \quad (7)$$

c_k represents the average behavior pattern of all nodes under the meta-path p_k , and γ_k represents the global contribution of the meta-path p_k to the classification task.

Then, our method employs linear projection coupled with multi-head attention computation to capture complex

interaction patterns across diverse meta-path combinations. W_Q, W_K, W_V are learnable parameters and we set $M = 4$.

$$H_{\text{weighted}} = \sum_{k=1}^4 \gamma_k H_k \quad (8)$$

$$Q = H_{\text{weighted}} W_Q, \quad K = H_{\text{weighted}} W_K, \quad V = H_{\text{weighted}} W_V \quad (9)$$

$$\text{Attention}_m = \text{softmax} \left(\frac{Q_m K_m^\top}{\sqrt{d_h/M}} \right) V_m \quad (10)$$

The attention outputs are then aggregated to produce the final semantic-level embedding:

$$Z = \text{LayerNorm}([\text{Attention}_1 \parallel \dots \parallel \text{Attention}_M] W_O + H_{\text{weighted}}) \quad (11)$$

3) *Classification*: The model is trained by learning the final embeddings of labeled sample nodes using a cross-entropy loss function:

$$L = - \sum_{i \in \mathcal{Y}_L} Y_i \ln(\Theta \cdot Z^i) \quad (12)$$

where Θ denotes the model parameters, Z^i represents the embedding of labeled nodes and Y_i indicates the labels of marked nodes. Through minimizing this cross-entropy loss, the model learns to predict labels for unmarked nodes in the graph.

IV. EXPERIMENT SETTINGS

In this section, we provide preparation for the experiment. First, we specify the targets for monitoring and details in active probing. Besides, we also provide the method for acquiring groundtruth. Second, we describe the metrics used in the later experiment.

We use PyTorch [27] to run experiments on a server equipped with an NVIDIA Tesla V100 GPU with 16GB memory, an Intel Xeon Gold 6226R CPU (16 cores), and 128 GB RAM. For data collection, we deploy a scanner running on an Ubuntu 22.04 system with an Intel Xeon Platinum 8255C CPU (2 cores) and 2GB memory.

A. Active Probing

To acquire records of domain name from resolvers, we employ active probing for collection. Notably, we treat DNS servers serving dual roles (both resolver and authoritative) solely as public resolvers for our analysis.

The public resolver list is obtained from multiple sources [28], [29], [30], [31], [32]. In order to maximize the coverage, our principle is to choose the resolver ISP public resolvers and famous national DNS services [33]. Nevertheless, there also exist invalid resolvers. Hence, we adopt a two-stage validation process to filter out unreliable services following the two processes: (1) Port verification using geographically distributed scanners to confirm open port 53 accessibility, and (2) DNS query validation requiring correct *NOERROR* responses to controlled test queries. These criteria effectively filter out private resolvers and authoritative-only servers. Additionally, we utilize active domains from Tranco [34], including

government, critical infrastructure, and sensitive domains. This diverse dataset enables comprehensive DNS hijacking analysis.

We deploy five globally distributed scanners to conduct continuous DNS measurements over 10 months (September 2024 to April 2025), with detection rounds performed every 2–3 days. Each round targeted 2,001 public resolvers and 36,192 domains. For this study, we analyze data from the latest seven rounds, with each round generating approximately 20M packets. In probing, we use ZDNS [35] as the tool to send queries. Notably, to avoid potential ethical risk and disruptions to DNS service, we strictly limit the query rate and randomize the query order.

To support evaluation, groundtruth of hijacking events is manually confirmed from multiple sources. Specifically, hijacking samples are identified using threat intelligence platforms, including VirusTotal and ThreatBook [36], [37]. Additionally, we develop a crawler that uses custom host headers to access web pages from answer_ips. By comparing the content with trusted References, we can identify potential DNS hijacking. For benign samples, we configure a controlled local recursive resolver and use its responses as trusted data. We exclude ambiguous cases (responses that could not be definitively classified as hijacking or benign) from both training and testing datasets. Our method incorporates auxiliary IP-ASN and IP-geolocation data obtained from the MaxMind database [25].

B. Ethical Considerations

To minimize potential impact on DNS infrastructure, we strictly limit the query rate and randomize the resolver selection. These measures ensure that no individual resolver or authoritative server experiences significant load. Furthermore, our experiments do not involve any user-side data collection. We do not access or analyze personal DNS traffic, and the entire dataset is generated from controlled queries.

C. Metrics

For a comprehensive evaluation, we use several standard metrics derived from the confusion matrix, including Accuracy (ACC), Precision, Recall, F1 score, Balanced Accuracy (B-Acc), Cohen's kappa, Matthews correlation coefficient (MCC), and Area Under the ROC Curve (AUC). Since the dataset is imbalanced, we pay particular attention to B-Acc and kappa. The formulas for these two metrics are as follows:

$$\text{B-Acc} = \frac{1}{2} \left(\frac{\text{TP}}{\text{TP} + \text{FN}} + \frac{\text{TN}}{\text{TN} + \text{FP}} \right) \quad (13)$$

$$\begin{aligned} \text{Kappa} &= \frac{p_o - p_e}{1 - p_e}, p_o = \frac{\text{TP} + \text{TN}}{N}, \\ p_e &= \frac{(\text{TP} + \text{FP})(\text{TP} + \text{FN})}{N^2} + \frac{(\text{TN} + \text{FN})(\text{TN} + \text{FP})}{N^2} \end{aligned} \quad (14)$$

TN indicates the number of benign answer_ips correctly classified; FP indicates the number of benign answer_ips classified as hijacked; FN indicates the number of hijacked answer_ips classified as benign; TP indicates the number of hijacked answer_ips correctly classified.

TABLE IV
PERFORMANCE OF HINHJ

Label Proportion	Accuracy	Kappa	Recall	Precision	MCC	AUC
90%	0.9997	0.9917	0.9839	0.9998	0.9918	0.9997
70%	0.9992	0.9719	0.9875	0.9576	0.9720	0.9963
50%	0.9989	0.9639	0.9663	0.9627	0.9639	0.9967
30%	0.9984	0.9453	0.9385	0.9538	0.9453	0.9955
10%	0.9978	0.9246	0.9064	0.9458	0.9248	0.9970

V. EXPERIMENTS

In this section, we present comprehensive experiments to evaluate HINHJ. In total, we conduct three experiments for evaluation. First, we show the overall performance of HINHJ as an initial evaluation. Second, we compare HINHJ with baseline detection methods from the perspective of precision. Third, we provide visualization analysis demonstrating our dual-layer attention mechanism’s effectiveness in identifying important nodes and meta-paths. Fourth, we describe several hijacking cases detected by HINHJ.

A. Preliminary Experiment of HINHJ

To evaluate the model’s robustness and performance under varying levels of supervision, we simulate scenarios with different amounts of labeled data. Specifically, we select $k\%$ of graph nodes as training samples, where k is set to 10, 30, 50, 70, or 90. By varying k , we are able to assess the model’s behavior across a spectrum of labeled data availability. HINHJ still needs a validation set to train the parameters of the model, so we then select $m\%$ (where m is equal to 10) of the dataset as the validation set, using the remaining nodes for testing model performance.

Table IV and Table V show the overall experiment results when using different ratios of label proportion. It can be seen that HINHJ achieves strong performance when trained on 90% of graph nodes, reaching 99% F1-score. This indicates our method performs exceptionally well with sufficient training data. As the number of training samples decreases, there is a small decrease in each evaluation metric value of HINHJ. When HINHJ is trained with only a small number of training label samples (about 10% of training samples in the graph), it also has good performance with balanced accuracy above 95%. These results demonstrate that our graph structure features and two-layer attention network effectively extend detection coverage to identify related hijacked nodes.

It can be seen from Table IV that our method maintains stable accuracy as label proportions decrease. However, recall declines slightly due to behavioral variations across different hijacking categories and organizations. HINHJ misses some hijacking cases where attackers use reputable cloud servers. These servers exhibit normal behavior in other periods, leading to false negatives. Additionally, some false positives occur with geographically optimized domain resolutions that show mutation in timing characteristics.

Moreover, considering the cost of labeling data and detection efficiency in practical applications, we evaluate the method’s F-score and loss at 50% label proportion as shown in Fig. 3. The performance improves with increasing epochs and stabilizes at epoch 100, which we consequently adopt for all subsequent experiments. The corresponding training time

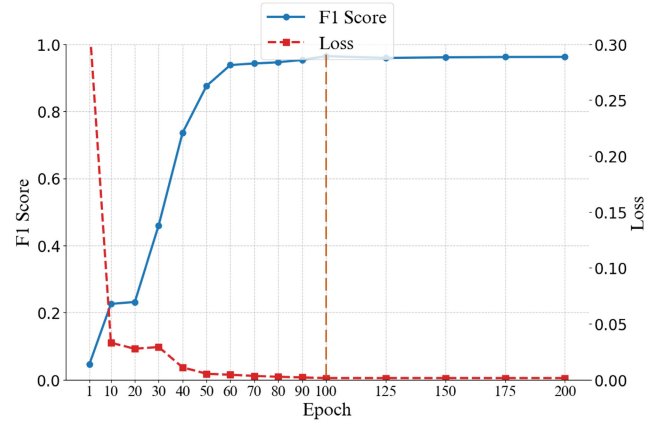


Fig. 3. Training loss and F1-score across epochs. The model quickly converges within 100 epochs, with loss decreasing and F1-score stabilizing above 0.95.

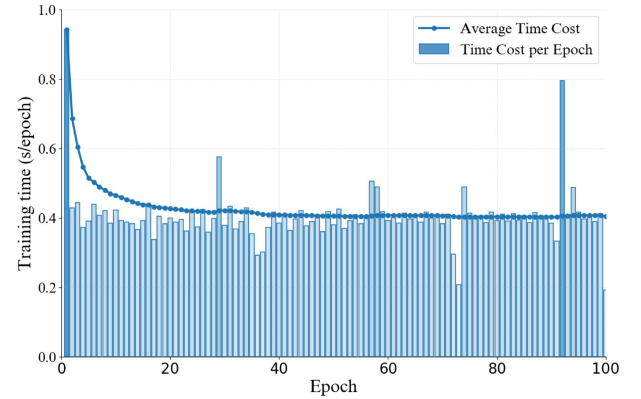


Fig. 4. Training time per epoch across epochs. The bars illustrate the training time for each epoch, while the line represents the cumulative average training time.

per epoch is illustrated in Fig. 4, showing that each epoch takes less than 0.5 seconds on average, indicating high efficiency.

Further, we find that some false alerts in HINHJ are caused by legitimate answer_ip connected to hijacked ones via meta-paths. This typically happens when attackers use shared infrastructure such as CDN nodes or cloud services for malicious activities. However, such cases are relatively rare and do not significantly affect the overall performance of our method.

B. Comparison Experiments

To evaluate HINHJ, we compare it with both traditional machine learning models and recent state-of-the-art methods. For traditional machine learning methods, we use RF, XGBoost, and AdaBoost, and apply the node features designed by HINHJ to these models. In addition, we introduce several state-of-the-art approaches: (1) MC-RF and MC-GBDT [15], machine learning methods detecting hijacking through multi-day DNS record analysis; (2) G-Belief [18], a semi-supervised method based on graph modeling answer_ip-domain relationships with SybilScar belief propagation [38] for reputation scoring; (3) SeHGNN [39], a simple and efficient heterogeneous GNN framework that aggregates neighborhood information using a parameter-free design tailored for heterogeneity; (4) LMSPS [40], a graph learning method that

TABLE V

PERFORMANCE COMPARISON ACROSS DIFFERENT LABEL PROPORTIONS. NF INDICATES THE USE OF OUR EXTRACTED NODE FEATURES; HG DENOTES THE USE OF OUR CONSTRUCTED HETEROGENEOUS GRAPH; MP INDICATES THE USE OF OUR MANUALLY DEFINED META-PATHS

Label Proportions	10%		30%		50%		70%		90%	
Methods	F1	B-Acc	F1	B-Acc	F1	B-Acc	F1	B-Acc	F1	B-Acc
RF+NF	0.7459	0.7974	0.6326	0.7328	0.6970	0.7689	0.6973	0.7260	0.9385	0.9607
XGBoost+NF	0.7691	0.8109	0.7693	0.8143	0.9024	0.9131	0.8789	0.8951	0.7292	0.9760
AdaBoost+NF	0.7486	0.7994	0.7667	0.8125	0.8655	0.8864	0.9017	0.9129	0.9303	0.9734
MC-RF	0.3993	0.6065	0.4149	0.6140	0.4348	0.6335	0.4414	0.6594	0.4349	0.6517
MC-GBDT	0.3853	0.5844	0.3747	0.5798	0.3999	0.5915	0.3974	0.5873	0.4084	0.6047
G-Belief	0.7224	0.8102	0.7800	0.8401	0.8150	0.8845	0.8502	0.8693	0.8995	0.8998
SeHGNN+HG+MP	0.8446	0.8812	0.8333	0.8740	0.9115	0.9434	0.9651	0.9749	0.9623	0.9808
LMSPS+HG	0.7394	0.7852	0.7273	0.8460	0.8303	0.8796	0.8922	0.9365	0.9512	0.9655
TESA+HG	0.7009	0.7921	0.7412	0.8953	0.8904	0.9117	0.9223	0.9405	0.9607	0.9672
HINHJ	0.9257	0.9528	0.9461	0.9689	0.9645	0.9829	0.9723	0.9934	0.9919	0.9919

enhances long-range dependency modeling on large-scale heterogeneous graphs by searching informative meta-paths across multiple hops. (5) TESA [41], a temporal heterogeneous GNN that captures temporal behavioral patterns via trajectory-based evolution and models relation-specific semantics through semantic-aware aggregation.

Regarding input requirements, MC-RF, MC-GBDT, and G-Belief operate directly on DNS records or answer_ip-domain graphs and do not require manually constructed heterogeneous graphs or meta-paths. SeHGNN requires a heterogeneous graph together with manually specified meta-paths, both supplied by our constructed graph. In contrast, LMSPS performs automatic meta-path discovery and does not rely on our manually defined meta-paths. For TESA, it directly captures the historical behavior of answer_ip nodes using a timestamped heterogeneous graph without the need for meta-paths.

Considering the significant imbalance of benign and hijacking instances (1% hijacking messages in ground truth), we primarily assess performance using F1 and B-Acc as shown in Table V. These results demonstrate that general-purpose models achieve good performance when combined with our extracted node features (NF), constructed heterogeneous graph (HG), and manually defined meta-paths (MP). This confirms the effectiveness of our feature extraction, graph construction, and meta-path design.

First, when compare with traditional machine learning based methods, HINHJ consistently outperforms machine learning baselines across all label proportions. Notably, XGBoost and AdaBoost achieve more than 85% F1 at 50% label proportion, demonstrating that our features effectively capture DNS hijacking behavior and approximate node graph structures. However, their graph representation capability remains inferior to graph models, explaining HINHJ's superior performance. Moreover, XGBoost and AdaBoost outperform other machine learning methods. This stems from our high-dimensional feature space containing numerous sparse, nonlinearly separable features, which particularly disadvantages linear classifiers.

Second, when compare with state-of-the-arts, it can be seen that the F1 and B-Acc of our method are significantly higher than all other methods. In addition, the performance of MC-RF and MC-GBDT methods is lower than the other four machine learning methods using our extracted features, and the

performance of G-Belief method is lower than XGBoost and AdaBoost. These results prove that the features we designed can well mine DNS hijacking behavior patterns.

Both SeHGNN and LMSPS operate on the heterogeneous graph constructed by our method. However, neither method incorporates time-aware neighbor information, which limits their ability to characterize temporal resolution behaviors. For SeHGNN, manually specified meta-paths are required. We therefore provide the same meta-paths used in HINHJ. LMSPS, in contrast, selects meta-paths automatically, but the selected ones do not match DNS hijacking well. As a result, SeHGNN achieves better performance than LMSPS. In comparison, HINHJ integrates temporal modeling with our designed meta-paths and node-level attention, leading to consistently superior performance. TESA operates on a temporal heterogeneous graph and captures the time-evolving behavior of individual answer_ip nodes. However, it overlooks semantic correlations across nodes, which are crucial in DNS hijacking detection for identifying coordinated behaviors. HINHJ addresses this by leveraging meta-paths to aggregate cross-node semantics, leading to more expressive and discriminative representations. As a result, TESA shows weaker performance compared to HINHJ.

When we deep into the falsely predicted instances, we found that MC-RF/MC-GBDT generate false positives from legitimate DNS variations, like dynamic IP, and miss long-term hijacking (e.g., censorship) by only examining temporal resolver behavior. G-Belief's graph propagation fails when hijacking affects few domains as it lacks supplementary statistical features. This explains why our feature design, which combines graph structures with statistical features, achieves superior performance when applied to machine learning methods compared to these baseline approaches. SeHGNN and LMSPS effectively capture answer_ip associations in the daily graph, but due to the lack of historical context modeling, they miss some long-term hijacking cases. Due to the lack of modeling correlations between answer_ip nodes, TESA misses some hijacking cases where suspicious answer_ips resolve the same domain or resolvers.

Our proposed TNHAN is designed based on HAN [42], [43], [44] by introducing a temporal neighbor mechanism and a multi-head attention mechanism at the semantic layer. To

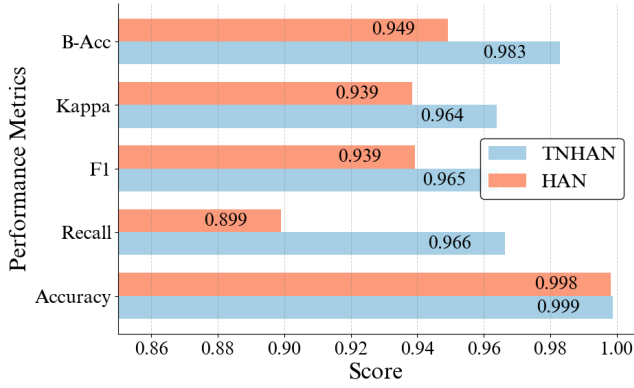


Fig. 5. Performance evaluation on HINHJ and TNHAN. TNHAN introduces a temporal neighbor selection mechanism based on the HAN model to improve the ability to capture temporal neighbor nodes.

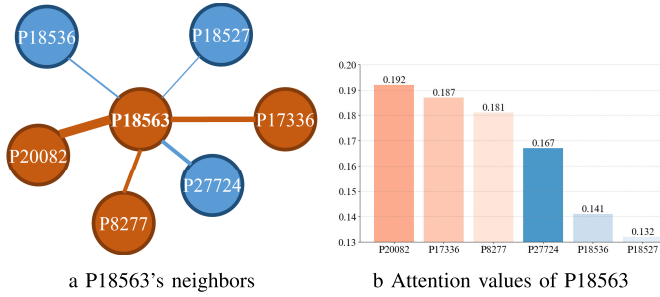


Fig. 6. Neighbors and attention values of node P18563. Red represents DNS hijacking nodes, and blue represents normal nodes.

verify the effect of our improvement on the model, we compare the proposed model with the original HAN model [42], and the results are shown in Fig. 5. It can be seen that our method can capture the temporal neighbor node information of the node, so it can achieve higher performance than HAN.

C. Analysis of Hierarchical Attention

Our approach hierarchically integrates neighboring node relationships and multi-relational meta-paths. To facilitate a deeper understanding of its decision process, we visualize both node-level attention and semantic-level attention.

As aforementioned, HINHJ effectively assigns attention weights among neighboring nodes within meta-paths, where critical neighbors for detection receive higher attention values. Fig. 6 demonstrates this mechanism through P18563's neighborhood analysis. In sub-Figure 6a, P18563 connects to six nodes - three hijacked (P20082, P27724, P18563) and three normal (P18563, P18563, P27724). Edge widths represent attention weights, revealing significantly stronger attention (thicker edges) to the hijacked nodes than to normal ones. This selective attention mechanism enables our model to automatically prioritize relevant nodes while suppressing irrelevant connections.

As shown in Fig. 7, our analysis of meta-path attention across varying label proportions reveals P3 and P4 are more important compared to P1 and P2, as shown in Fig. 7. This is mainly because the DNS hijacking we detect occurs on the public resolver, and the hijacker tends to hijack different

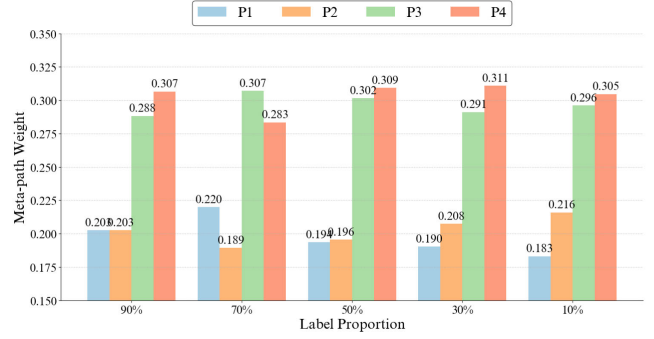


Fig. 7. Meta-path attention for different label proportion.

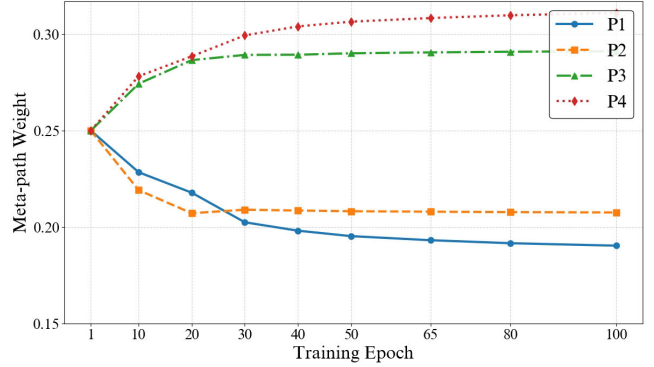


Fig. 8. The change in attention of different meta-paths as the training epoch increases.

domains on the same resolver. These hijackings point to IPs that share the same resolver.

In addition, we further record the weight evolution of meta-paths throughout training, as shown in Fig. 8. Initially uniform, the weights diverge as training progresses: P1 and P2 decline, while P3 and P4 rise. This shows our method's ability to dynamically discern meta-path importance for DNS hijacking detection.

D. Analysis of Time Decay Mechanism

Our method uses a time decay mechanism to model the historical connections between nodes, where more recent neighbor relations are assigned higher weights through a decay factor λ . A larger λ means historical neighbors have more influence. For each candidate neighbor, we compute a cumulative weight by summing its past occurrences, each discounted by a factor of λ based on its recency. Only neighbors whose total weight exceeds the threshold are kept. This helps the model focus on recent patterns while reducing the effect of outdated or rare connections.

To evaluate the effect of the time decay factor λ , we vary its value from 0 to 0.9 and evaluate the F1-scores, as shown in Fig. 9. Notably, $\lambda = 0$ corresponds to the no-decay setting, where the model only uses neighbors from the daily subgraph, resulting in an F1-score of 0.939. As λ increases, the model includes more historical neighbors, leading to improved performance. The F1-score peaks at $\lambda = 0.7$, suggesting that leveraging historical information benefits detection. However, setting λ too high brings in outdated neighbors, which introduces noise and degrades performance.

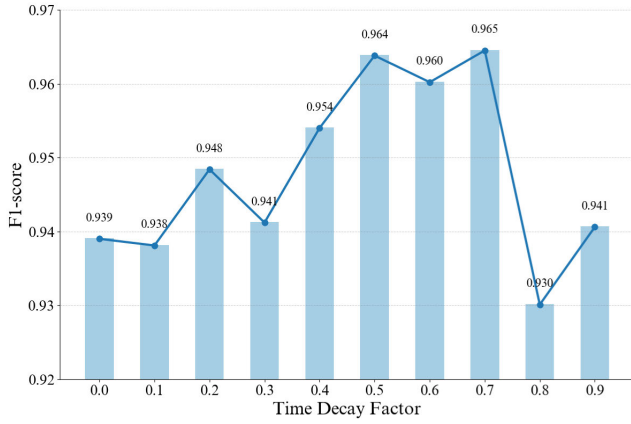


Fig. 9. F1-score under different time decay factors λ . The performance peaks at time decay factor $\lambda = 0.7$.

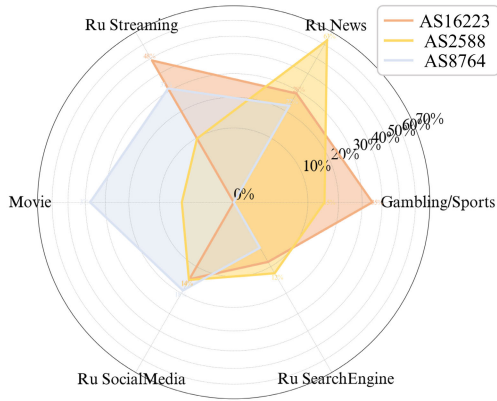


Fig. 10. The proportion of domains hijacked by AS16223, AS8764 and AS2588, with Russian domains accounting for the majority.

E. Real-World Dns Hijacking Incidents

To illustrate this approach concretely, we first describe how we use HINHJ to identify a set of DNS hijacking incidents in reality. Then we discuss the characteristics of the full set of hijacking events and the validation. In general, we apply HINHJ to real-world networks and discover multiple types of hijacking incidents, including geopolitical conflicts, DNS censorship, malicious hijacking, and configuration errors.

1) *Geopolitical Conflicts*: As a concrete example of our method, we discovered a DNS hijacking activity on Ukrainian resolver AS16223. Compromised IPs associated with this resolver exhibit strong meta-path correlations with known malicious infrastructure through meta-paths P3 and P4 while exhibiting significant deviations from benign IPs in activity features and security features. While most of its domain resolutions appear legitimate, our method identifies targeted hijacking of 70 specific domains—29 *.ru* TLDs (e.g., *yandex.ru*) and 41 Russian-oriented non-*.ru* sites (e.g., *kinogo.best*)—redirecting traffic to Ukrainian IPs within AS16223. As shown in Fig. 10, the website types include search engine websites, streaming media websites, news websites, social networking websites, etc.

Through manual verification with extra information, we confirm DNS hijacking for Russian domains on AS16223. The Ukrainian-operated host *columbus.te.ua* presents parking

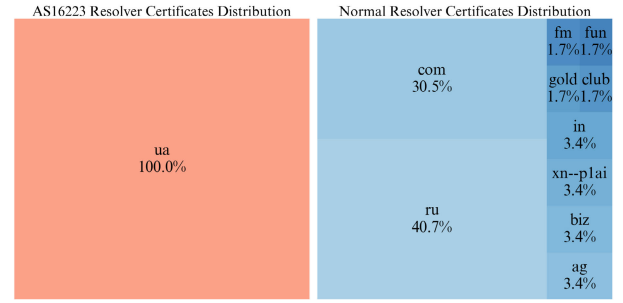


Fig. 11. Certificate comparison for hijacking answer_ip (AS16223) and normal answer_ip. Red markings indicate the hijacking answer_ip (AS16223) with all certificates having the *.ua* TLD, blue markings denote normal answer_ips with certificates including *.com*, *.ru*, *.ag*, and other TLDs but excluding *.ua*.

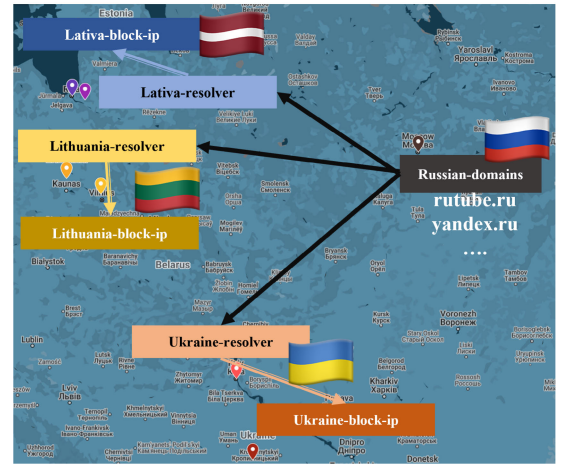


Fig. 12. Hijacking cases of some Russian domains (e.g., *rutube.ru*, *yandex.ru*). Public resolvers in Latvia, Lithuania, and Ukraine are shown to resolve Russian domains in their respective countries.

certificates *parking.columbus.te.ua* for these domains, contrasting sharply with normal regional resolution certificates shown in Fig. 11. This discrepancy rules out legitimate CDN optimization. Furthermore, normal resolvers maintain proper *.ru*-authoritative NS records (e.g., *ns1.yandex.ru* for *yandex.ru*) while AS16223 resolver does not. This provides conclusive evidence of DNS hijacking rather than infrastructure anomalies.

Our findings demonstrate that the Ukrainian AS16223 public resolver has been hijacking Russian domains. This pattern extends to other regional public resolvers - Lithuania's AS8764 and Latvia's AS2588 similarly target *.ru* domains and Russian news sites as shown in Fig. 12 and Fig. 10, aligning with their governments' political positions. Notably, while most countries' public resolvers hijack both illegal content and politically sensitive domains, Lithuania's AS8764 exclusively targets Russian domains and excludes illegal domains such as gambling, reflecting a focused political agenda rather than standard security filtering.

In contrast, we found that a number of public resolvers also hijack *.ua* domains, including Russia, Belarus and Iran, mainly including Ukrainian news websites *nv.ua*, *unian.ua*, *focus.ua*, etc. The relevant manual judgment process is similar to the previous one.

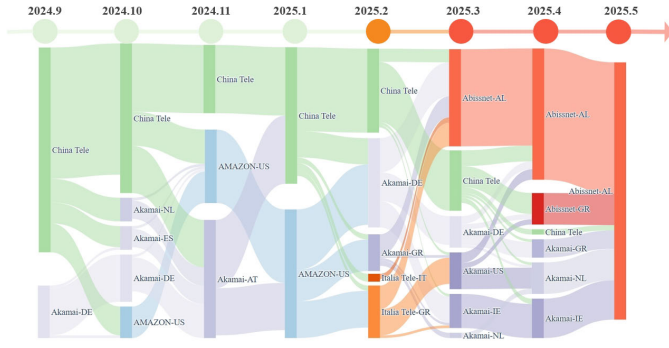


Fig. 13. Changes in the answer_ip organization of TikTok-related domains resolved by the Albanian resolver. In March 2025, Albania decided to block TikTok, and the proportion of abissnet (affiliated with the Albanian government) in answer_ip organizations gradually increased. In February 2025, there was a resolution to Italia-Tele, speculated to be a pre-blocking test.

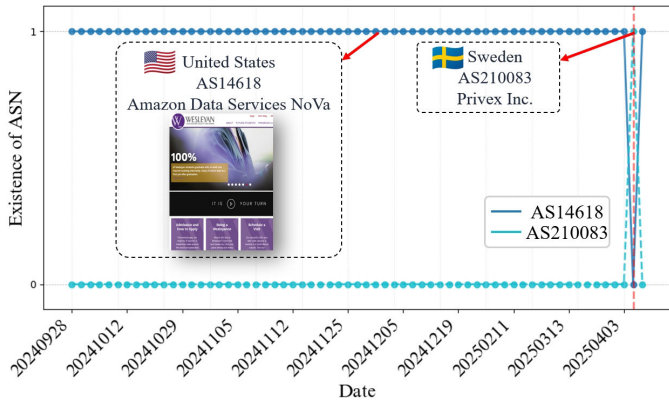


Fig. 14. Changes in the AS50057 resolver's resolution results for the domain wesleyancollege.edu. The domain was hijacked for less than a day.

2) *Censorship of ByteDance-Related Domains*: In April 2025, our method detected DNS hijacking of ByteDance domains (including *tiktok.com*, *bytedance.com*, etc.) by Albania's public resolver AS35037. Verification through certificates and landing pages confirms this as hijacking rather than CDN usage. As shown in Fig. 13, historical analysis since September 2024 reveals stable resolution patterns through China Telecom, Akamai, and Amazon cloud servers until February 2025, when *tiktok.com* first redirected to Italia Telecom. The resolution pattern shifted dramatically in March 2025, with Albania-local servers (matching the resolver's AS) gradually replacing previous destinations, showing a systematic takeover rather than temporary anomalies.

News reports confirm that the Albanian government officially banned TikTok in March 2025 for alleged legal violations [45], coinciding with our detection of ByteDance domains being redirected to Abissnet servers. This indicates Albania's censorship strategy includes DNS hijacking through public resolvers. The earlier redirection to an Italian server on February 15, 2025 suggests testing of DNS blocking mechanisms, demonstrating that ByteDance restrictions were already being implemented at that time.

3) *Malicious Hijacking*: Our method also detected short-lived hijacking events, such as a less-than-one-day hijack of *wesleyancollege.edu* by Iran's AS50057 resolver in April 2025, redirecting to AS210083. As shown in Fig. 14, analysis

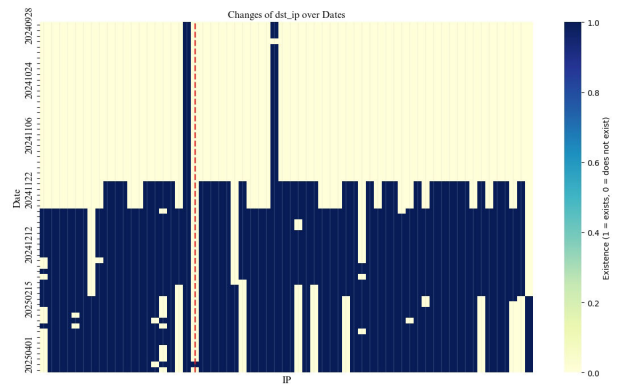


Fig. 15. Answer_ip distribution of wesleyancollege.edu resolution results on all resolvers. Dark shades indicate the presence of records on the corresponding date, while light shades indicate no records.

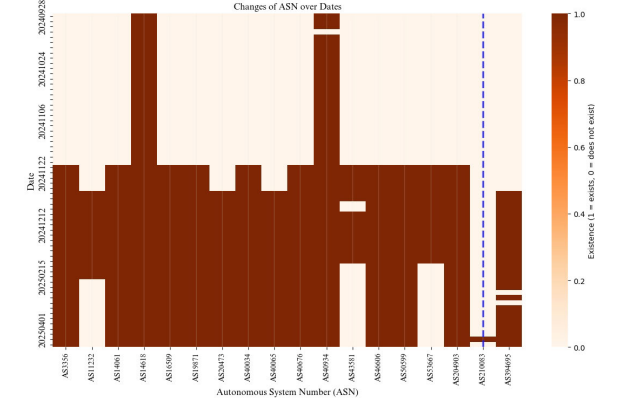


Fig. 16. ASN distribution of wesleyancollege.edu resolution results on all resolvers. Dark shades indicate the presence of AS on the corresponding date, while light shades indicate no records.



Fig. 17. Abnormal response message about name-mismatch.

of the domain's resolution history shows stable patterns except for this brief anomaly on April 8. Comparative resolution distributions across all resolvers confirm temporal consistency in most answer_ip assignments, with AS210083 appearing only during the attack window as shown in Fig. 15 and Fig. 16. Certificate verification and threat intelligence checks identify the answer_ip as malicious, collectively proving this was a deliberate DNS cache poisoning attack rather than CDN-related anomalies. It is worth noting that *wesleyancollege.edu* is a women's university and this hijacking may be relevant to the situation of women in Iran.

4) *Configuration Error*: In addition to the hijacking incidents mentioned above, our approach also discovers some records with abnormal resolver configurations. We find that there was a mismatch between the name in the response A record and the requested domain, which has not been studied before.

Our method identifies a novel resolver misconfiguration termed "name-mismatch", where queries return correct answer_ip but incorrect domain names (e.g., *ahcme.edu.cm*

resolving to elster.de's IP, as shown in Fig. 17). Analysis of 2M messages from April 8, 2025 reveals 95 such cases across 8 resolvers and 94 domains. Daily detection shows 100-200 name mismatch occurrences with varying resolver lists, while a few resolvers exhibit this behavior persistently with different domains each day. This pattern suggests underlying software/hardware defects rather than manual configuration errors.

VI. CONCLUSION

In this paper, we propose an effective method HINHJ for detecting DNS hijacking. HINHJ first constructs a heterogeneous graph to represent the cross-entity association relationship between domains, answer_ips, and resolvers, and then extracts features based on the behavior patterns of each answer_ip. In order to better combine graph structure features and statistical features, HINHJ uses an improved HAN model TNHAN to learn the embedding representation of each answer_ip through node-level and semantic-level attention networks. We evaluate HINHJ on the test set and in the real network environment. The experimental results show that our method can achieve accurate detection performance when compared with other state-of-the-arts. Besides, we also provide several DNS hijacking cases in real-world.

REFERENCES

- [1] R. Arends, R. Austein, M. Larson, D. Massey, and S. Rose, "DNS security introduction and requirements," RFC Editor, Fremont, CA, USA, Tech. Rep. RFC 4033, 2005.
- [2] P. Pearce et al., "Global measurement of DNS manipulation," in *Proc. 26th USENIX Secur. Symp. (USENIX Secur.)*, 2017, pp. 307–323.
- [3] A. Bhaskar and P. Pearce, "Understanding routing-induced censorship changes globally," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Dec. 2024, pp. 437–451.
- [4] L. Jin, S. Hao, H. Wang, and C. Cotton, "Understanding the impact of encrypted DNS on internet censorship," in *Proc. Web Conf.*, Apr. 2021, pp. 484–495.
- [5] J. Park, M. Mohaisen, and A. Mohaisen, "Investigating DNS manipulation by open DNS resolvers," in *Proc. 15th Int. Conf. Emerg. Netw. Experiments Technol.*, Dec. 2019, pp. 45–46.
- [6] K. Man, X. Zhou, and Z. Qian, "DNS cache poisoning attack: Resurrections with side channels," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2021, pp. 3400–3414.
- [7] K. Man, Z. Qian, Z. Wang, X. Zheng, Y. Huang, and H. Duan, "DNS cache poisoning attack reloaded: Revolutions with side channels," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2020, pp. 1337–1350.
- [8] X. Li et al., "The maginot line: Attacking the boundary of DNS caching protection," in *Proc. 32nd USENIX Secur. Symp. (USENIX Secur.)*, 2023, pp. 3153–3170.
- [9] Y. Afek, A. Bremner-Barr, and L. Shafir, "NXNSAttack: Recursive DNS inefficiencies and vulnerabilities," in *Proc. 29th USENIX Secur. Symp. (USENIX Secur.)*, 2020, pp. 631–648.
- [10] A. Randall et al., "Home is where the hijacking is: Understanding DNS interception by residential routers," in *Proc. 21st ACM Internet Meas. Conf.*, Nov. 2021, pp. 390–397.
- [11] M. Luo, L. Xin, Y. Yao, Z. Jiang, Q. Wang, and W. Shi, "Who are querying for me? Egress measurement for open DNS resolvers," in *Proc. 26th Int. Conf. Comput. Supported Cooperat. Work Design (CSCWD)*, May 2023, pp. 1544–1550.
- [12] G. Akiwate et al., "Retroactive identification of targeted DNS infrastructure hijacking," in *Proc. 22nd ACM Internet Meas. Conf.*, Oct. 2022, pp. 14–32.
- [13] Y. Nosyk et al., "Intercept and inject: Dns response manipulation in the wild," in *Proc. Int. Conf. Passive Act. Netw. Meas.* Cham, Switzerland: Springer, 2023, pp. 461–478.
- [14] B. Liu et al., "Who is answering my queries: Understanding and characterizing interception of the DNS resolution path," in *Proc. 27th USENIX Secur. Symp. (USENIX Secur.)*, 2018, pp. 1113–1128.
- [15] R. Houser, S. Hao, Z. Li, D. Liu, C. Cotton, and H. Wang, "A comprehensive measurement-based investigation of DNS hijacking," in *Proc. 40th Int. Symp. Reliable Distrib. Syst. (SRDS)*, Sep. 2021, pp. 210–221.
- [16] M. Fejrskov, J. M. Pedersen, and E. Vasilomanolakis, "Detecting DNS hijacking by using NetFlow data," in *Proc. IEEE Conf. Commun. Netw. Secur. (CNS)*, Jul. 2022, pp. 273–280.
- [17] Y. Jin, M. Tomoishi, and S. Matsuura, "A detection method against DNS cache poisoning attacks using machine learning techniques: Work in progress," in *Proc. IEEE 18th Int. Symp. Netw. Comput. Appl. (NCA)*, Sep. 2019, pp. 1–3.
- [18] J. Jia, Z. Dong, J. Li, and J. W. Stokes, "Detection of malicious DNS and web servers using graph-based approaches," in *Proc. IEEE Int. Conf. Acoust. (ICASSP)*, Aug. 2021, pp. 2625–2629.
- [19] WebRAY & Panabit. (2024). *Analysis of a Large Number of Abnormal Network Access and Traffic Hijacking Incidents of Domestic Home Routers*. [Online]. Available: <https://blog.csdn.net/c20220511/article/details/141019242>
- [20] Google Public DNS. Accessed: Apr. 23, 2025. [Online]. Available: <https://dns.google.com/>
- [21] A. Grover and J. Leskovec, "Node2vec: Scalable feature learning for networks," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2016, pp. 855–864.
- [22] K. Govindan, K. Arunachalam, and K. Subramaniam, "Optimal server selection policy for improved network efficiency in smart phones," in *Proc. IEEE Wireless Commun. Netw. Conf. (WCNC)*, Apr. 2018, pp. 1–6.
- [23] K. Schomp, O. Bhardwaj, E. Kurdoglu, M. Muhaimen, and R. K. Sitaraman, "Akamai DNS: Providing authoritative answers to the world's queries," in *Proc. ACM Conf. Spec. Interest Group Data Commun. (SIGCOMM)*, 2020, pp. 465–478.
- [24] Y. Sun, Y. Yu, and J. Han, "Ranking-based clustering of heterogeneous information networks with star network schema," in *Proc. 15th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2009, pp. 797–806.
- [25] MaxMind Inc. (2025). *GeoIP2/GeoLite2 Database*. [Online]. Available: <https://www.maxmind.com>
- [26] C. Shi, Y. Li, J. Zhang, Y. Sun, and P. S. Yu, "A survey of heterogeneous information network analysis," *IEEE Trans. Knowl. Data Eng.*, vol. 29, no. 1, pp. 17–37, Jan. 2017.
- [27] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 8026–8037.
- [28] Trickest. (2024). *Resolvers: The Most Exhaustive List of Reliable DNS Resolvers*. [Online]. Available: <https://github.com/trickest/resolvers>
- [29] Cujanovic. (2023). *Public-DNS-Resolvers: List of Periodically Validated Public DNS Resolvers*. [Online]. Available: <https://github.com/cujanovic/public-dns-resolvers>
- [30] (2023). *Public DNS Server List*. [Online]. Available: <https://public-dns.info/>
- [31] Mutin-Sa. (2021). *Top Public Recursive Name Servers*. [Online]. Available: <https://gist.github.com/mutin-sa/5dcdb35ee436eb629db7872581093bc5>
- [32] M. Fejrskov, E. Vasilomanolakis, and J. M. Pedersen, "A study on the use of 3rd party DNS resolvers for malware filtering or censorship circumvention," in *Proc. IFIP Int. Conf. ICT Syst. Secur. Privacy Protection*, 2022, pp. 109–125.
- [33] A. Randall et al., "Trufflehunter: Cache snooping rare domains at large public DNS resolvers," in *Proc. ACM Internet Meas. Conf.*, Oct. 2020, pp. 50–64.
- [34] V. Le Pochat, T. Van Goethem, S. Tajalizadehkhoob, M. Korczyński, and W. Joosen, "Tranco: A research-oriented top sites ranking hardened against manipulation," 2018, *arXiv:1806.01156*.
- [35] L. Izhikevich et al., "ZDNS: A fast DNS toolkit for internet measurement," in *Proc. 22nd ACM Internet Meas. Conf.*, Oct. 2022, pp. 33–43.
- [36] (2025). *Virustotal: R Client for the Virustotal API*. [Online]. Available: <https://www.virustotal.com>
- [37] (2025). *Threatbook*. [Online]. Available: <https://x.threatbook.com>
- [38] B. Wang, L. Zhang, and N. Z. Gong, "SybilSCAR: Sybil detection in online social networks via local rule based propagation," in *Proc. IEEE Conf. Comput. Commun. (IEEE INFOCOM)*, May 2017, pp. 1–9.
- [39] X. Yang, M. Yan, S. Pan, X. Ye, and D. Fan, "Simple and efficient heterogeneous graph neural network," in *Proc. AAAI Conf. Artif. Intell.*, 2023, vol. 37, no. 9, pp. 10816–10824.

- [40] Z. Guo, K. He, Q. He, and C. Li, "Long-range meta-path search on large-scale heterogeneous graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 37, 2024, pp. 44240–44268.
- [41] X. Wang, J. Jiang, X. Yan, and Q. Huang, "Tesa: A trajectory and semantic-aware dynamic heterogeneous graph neural network," in *Proc. ACM Web Conf.*, 2025, pp. 1305–1315.
- [42] X. Wang et al., "Heterogeneous graph attention network," in *Proc. World Wide Web Conf.*, 2019, pp. 2022–2032.
- [43] M. Liu et al., "Enhanced detection of obfuscated HTTPS tunnel traffic using heterogeneous information network," *Comput. Netw.*, vol. 257, Feb. 2025, Art. no. 110975.
- [44] Q. Wang et al., "HANDOM: Heterogeneous attention network model for malicious domain detection," *Comput. Secur.*, vol. 125, Jun. 2022, Art. no. 103059.
- [45] CNN.(2025). *Albania Shuts Down Tiktok for a Year Amid Concerns Over Violence Among Children*. [Online]. Available: <https://edition.cnn.com/2025/03/06/europe/albania-shuts-down-tiktok-intl-latam/index.html>



Haoran Jiao received the M.S. degree from the School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China, in 2023. He is currently an Engineer with Zhongguancun Laboratory. His research interests include network security and network measurement.



Leyao NIE received the M.S. degree from the Institute for Network Sciences and Cyberspace, Tsinghua University, Beijing, China, in 2024. She is currently an Assistant Engineer with Zhongguancun Laboratory. Her current research interests include DNS measurement and situational awareness.



Yuling LIU received the Ph.D. degree from the Institute of Software, Chinese Academy of Sciences. He is currently a Professorate Senior Engineer with the Institute of Information Engineering, Chinese Academy of Sciences. His research interests include network security situational awareness, network security, big data analysis, and security measurement and certification.



Cong DONG received the Ph.D. degree from the School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China, in 2022. He is currently an Assistant Professor with Zhongguancun Laboratory. His research interests include network security awareness, network measurement, anomaly detection, and traffic analysis.



Jing WANG received the Ph.D. degree from Beijing University of Posts and Telecommunications, China, in 2012. She is currently an Associate Professor with the School of Artificial Intelligence, Beijing Normal University, China. Her current research interests include networking, communications, and security.



Chenglong LI received the B.E. and Ph.D. degrees in computer science and technology from Tsinghua University, China, in 2007 and 2012, respectively. He is currently an Associate Research Professor with the Institute for Network Sciences and Cyberspace, Tsinghua University. His research interests include network measurement, situational awareness, and the IoT security.



Changzhi ZHAO received the Ph.D. degree in cyber security from the School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China, in 2026. His research interests include network security awareness and anomaly detection.



Jiahai YANG (Senior Member, IEEE) received the B.Sc. degree in computer science from Beijing Technology and Business University and the M.Sc. and Ph.D. degrees in computer science from Tsinghua University, China. He is currently a Professor with the Institute for Network Sciences and Cyberspace and Beijing National Research Center for Information Science and Technology, Tsinghua University, Beijing, China. He is a Researcher Fellow with Zhongguancun Laboratory, Beijing. His research interests include network management, network measurement, network security, Internet routing, and cloud computing.



Xia YIN received the B.E., M.E., and Ph.D. degrees in computer science from Tsinghua University in 1995, 1997, and 2000, respectively. She is currently a Full Professor with the Department of Computer Science and Technology, Tsinghua University. Her research interests include future Internet architecture, formal method, protocol testing, and large-scale Internet routing.